DecisionLab — نبذة وخطة تحسين

فكرة المشروع

DecisionLab تطبيق يساعد المستخدم على اتخاذ قرار عبر مقارنة خيارات وفق معايير وأوزان يحددها، ثم تقييم كل خيار من 1 إلى 10. يحسب السيرفر الترتيب ويعرض الفائز وأسباب النتيجة.

المسار الأساسي: **تسجيل** ← **دخول** ← **إنشاء قرار** ← **خيارات ومعايير وأوزان** ← **تقييم** ← **نتائج** ← **تثبيت القرار أو إعادة فتحه للتعديل**.

المعمارية الحالية

المشروع تطبيق واحد مقسّم إلى وحدات، والفرونت والباك في مستودع واحد:

```
client/ — React + TypeScript + Vite + React Router
الصفحات والمكونات المشتركة
  AuthContext: المستخدم والتوكن واستعادة الجلسة
Bearer JWT وخدمات API: الطلبات
                ↓ HTTP / JSON
server/ — NestJS
  Controllers: استقبال الطلبات والتحقق من المدخلات
  Guards: الهوية والصلاحيات وملكية القرار
  Services: المستخدمون والقرارات والقوالب
  Calculation Engine: التحقق والحساب والترتيب والتفسير
                ↓ Mongoose
              MongoDB
```

- الصفحات: Login، Register، Decisions، New Decision، Workspace، Results، Templates.
- الحساب والترتيب يتمان بالسيرفر، والفرونت يعرض البيانات والنتائج.
- التسجيل يرجع مستخدماً، والدخول يرجع `accessToken` ومستخدماً. استعادة الجلسة عبر `GET /auth/me` .
- الحالات الأساسية للقرار: `Draft` و `Finalized` .
- نقاط القوة: تقسيم الوحدات، صلاحيات بالسيرفر، محرك حساب مختبر، وهوية بصرية مشتركة.
- الحدود الحالية: قراءة نتائج القرارات النهائية مباشرةً تحتاج استكمالاً، وبعض خطوات الحفظ والتقييم تحتاج توضيحاً، والنشر مؤجّل.

أولويات التحسين

نكمل رحلة المستخدم من إنشاء القرار لعرض النتيجة، ثم نضيف ميزات.

الأولوية	التحسين	الفائدة
1	إصلاح Results لقراءة نتائج القرار النهائي وفتحها مباشرةً	العودة للنتيجة بأي وقت
2	شرح أسباب تعطيل خانات وأزرار Workspace	معرفة الخطوة التالية
3	قائمة نواقص قبل الحساب: الخيارات، الأوزان، العلامات	إصلاح البيانات من الصفحة نفسها
4	حالات Unsaved changes / Saving / Saved وتنبيه قبل المغادرة	تقليل ضياع العمل
5	اختبار التسجيل والدخول والإنشاء والتقييم والنتائج وإعادة الفتح	كشف مشاكل الربط
6	استضافة الفرونت والباك وقاعدة البيانات	مشاركة المشروع وتجربته

أمثلة للتوجيه داخل Workspace، مع زر ينقل للقسم المطلوب:

> أضفت خيارات جديدة. احفظها أولاً لتبدأ التقييم.
>
> مجموع الأوزان 112%؛ خفّضه بمقدار 12 نقطة مئوية.
>
> بقيت 15 علامة لإكمال التقييم.

يُعرض سبب تعطيل Calculate تحته. وبعد استقرار التدفق، نحسّن عرض الفائز والترتيب ومساهمة المعايير، باستخدام بيانات السيرفر، ونضيف My decisions / All decisions لتوضيح الملكية.

تقسيم تنفيذ مقترح

هذا اقتراح تنسيق للتحسينات، وليس إعادة تعيين رسمية لمهام الفريق:

- **Hamid:** القائمة والإنشاء والمصادقة ورسائلها.
- **Yousef:** توجيه المستخدم في Workspace وتجربة Results.
- **فريق الباك:** قراءة نتائج Finalized والاتفاق على عقد الرد مع الفرونت.
- **الفريق معاً:** اختبار التدفق الكامل والنشر.

البداية المقترحة: **Results + وضوح Workspace**.

نصائح للبرزنتيشن

1. ابدأ بالمشكلة: «عندنا خيارات متعددة، وكل شخص عنده أولويات مختلفة؛ كيف نقارنها بشكل واضح؟» ثم عرّف الحل بجملة واحدة.
2. اعرض المعمارية بدقيقة: واجهة React → API في NestJS → MongoDB، مع الحساب والصلاحيات بالسيرفر.
3. استخدم مثالاً مفهوماً: شراء لابتوب، بثلاثة خيارات ومعايير مجموع أوزانها 100. حضّر علامات كاملة قبل العرض.
4. اعرض رحلة قصيرة: دخول → إنشاء من قالب → إضافة خيارات وحفظها → تقييم → Calculate → شرح الفائز ومساهمة المعايير.
5. وضّح أن الوزن هو أهمية المعيار، والعلامة هي جودة الخيار بالنسبة إليه. للمقارنة الحالية، استخدم معايير مصاغة بحيث تعني العلامة الأعلى أداءً أفضل؛ مثلاً «ملاءمة السعر» بدلاً من سعر خام.
6. لا تصف الفائز بأنه أفضل خيار مطلقاً؛ هو الأفضل وفق الأوزان والتقييمات المدخلة.
7. جهّز حساباً وقراراً مكتملين وقاعدة اختبار، وافحص السيرفر قبل العرض. احتفظ بصور أو فيديو احتياطي للتدفق.
8. كن دقيقاً بشأن المنفّذ والمؤجّل. لا تعتمد بعرضك على فتح رابط نتيجة Finalized مباشرةً قبل إصلاحه، ولا تصف النشر بأنه مكتمل.
9. اختم بتحسينين واضحين: استكمال قراءة النتائج وتحسين الإرشاد داخل Workspace. اجعل الكلام عن أثرهما على المستخدم.

أسئلة متوقعة: لماذا JWT؟ أين تُفحص الملكية؟ لماذا يجب أن تساوي الأوزان 100؟ ماذا يحدث عند التعادل؟ وما الفرق بين Draft وFinalized؟ راجع سلوك الكود الفعلي قبل الإجابة، خاصةً سياسة التعادل.